TUHH
Technische Universität Hamburg

Project Proposal

Childcare Center: Vacation Planing & How do I feel today

Prepared By

Rodrigo Coelho 651121 ; Rita Belo 651134 ; Matys Tavoso 651051; Raquel Santos 651006 ; Lara Gürbüz 651246

Content

1. Project overview 5
1.1. Project description 5
1.2. Team and workflow 6
1.3. Tools used 6
2. Vacation Planning 8
2.1. Problem statement/definition 8
2.1.1. Main problem statement 8
2.1.2. Sub-problems 8
2.1.3. Glossary of Terms 9
2.2. User stories analysis 11
2.2.1. Use Cases Analysis 13
2.2.2. Requirements 14
2.3. System design and architecture 15
2.3.1. Use Case Diagram 15
2.3.2. Class Diagram with OCL 16
2.3.3. EFSM (Extended Finite System Machine) Diagram 17
2.3.4. Activity Diagram 18
2.3.5. Sequence Diagram 20
2.3.6. Communication Diagram 20
2.3.7. TAPN (Timed-Arc Petri Net) Diagram 21
2.3.8. Architecture 24
2.4. Testing Approach/Strategy (test-suites) 25
2.4.1. Discussion on the topic 28
2.5. Interface design 29
3. How do I feel today 30
3.1. Problem statement 30
3.1.1. Main problem 30
3.1.2. Sub-problems 30
3.1.3. Glossary of terms 30
3.2. User-stories analysis 30

3.2.1. Use Cases Analysis ... 31
3.2.2. Requirements ... 32
3.3. System design and architecture ... 33
3.3.1. Use cases diagram ... 33
3.3.2. Class diagram ... 34
3.3.3. EFSM diagram ... 36
3.3.4. Activity diagram ... 37
3.3.5. Sequence diagram ... 38
3.3.6. Communication diagram ... 39
3.3.7. TAPN diagram ... 40
3.3.8. Architecture diagram ... 41
3.4. Testing approach/strategy (test-suite) ... 42
3.4.1. Discussion on the topic ... 43
4. Lessons learned ... 44
5. Conclusion and future work ... 45
6. Annexes ... 45

Table 1 - Actors for Vacation Planning ... 9
Table 2 - Main classes and Domain Concepts for Vacation Planning ... 10
Table 3 - Key Variables & Attributes for Vacation Planning ... 10
Table 4 - System Functions & Use Case Actions for Vacation Planning ... 11
Table 5 - User Stories for Vacation Planning ... 13
Table 6 - Use Cases for Vacation Planning ... 14
Table 7 - Test Suite using Gemini 2.5 Flash for Vacation Panning ... 28
Table 8 - User Stories for How do I feel Today ... 31
Table 9 - Use Cases for How Do I Feel Today ... 32
Table 10 - Test suits for How do I feel Today ... 43
Table 11- Text suites for Vacation Planning ... 49

Figure 1: Dienstplanung before ... 9
Figure 2: Anwesenheitsliste, before ... 9
Figure 3 - UML Use Case Diagram for Vacation Planning ... 16
Figure 4 - UML Class Diagram with OCL for Vacation Planning ... 17
Figure 5 - EFSM Diagram for Vacation planning ... 18
Figure 6 - Activity Diagram for Vacation Planning ... 19

Figure 7 - Sequence Diagram for Vacation Planning 20
Figure 8 - Communication Diagram for Vacation Planning 21
Figure 9 - Clearer view from TAPN for Vacation Planning 23
Figure 10 - TAPN Diagram for Vacation planning, using TAPAAL 24
Figure 11 - Architecture for Vacation Planning 25
Figure 12 - Vacation Request Interface 29
Figure 13 - Director Vacation Request Interface 29
Figure 14 - UML Use Case Diagram for How do I feel Today 34
Figure 15 - Use Class Diagram for How do I feel Today 36
Figure 16 - EFSM diagram for How do I feel Today 37
Figure 17 - Activity diagram for How do I feel Today 38
Figure 18 - Sequence diagram for How do I feel Today 39
Figure 19 - Comunication diagram for How do I feel Today 40
Figure 20 - TAPN diagram for How do I feel Today 41
Figure 21 - Architecture for How do I feel Today 42

1. Project overview

The goal of this project is to design a system that seamlessly handles absence planning and automatically adapts care schedules accordingly - Vacation Planning. In parallel, another system will focus on supporting children's well-being by tracking their emotional states through a feature called *"How do I feel today?"*, allowing staff to adjust daily routines and activities according to each child's emotional needs.

With this in mind, some of the main topics to be discussed are:

Problem:

Childcare centres often face staffing disruptions due to uncoordinated absences which leads to unbalanced workload, placing stress on employees risking lower care quality.

Solution:

A unified digital platform to manage leave requests, automate documentation workflows, detect and alert overlap defects, and provide optional daily children's emotional check-ins for staff. Inspired by the "How do I feel today?" initiative presented in the second video this component supports emotional transparency and responsive planning.

Scope:

The system covers vacation request submissions, director approvals, document validation, directors' alerts, emergency care plan coordination, and daily mood check-ins that can be visualized and acted upon, as and well-being self-reports.

Outcome:

A complete engineering documentation package including models (UML, EFSM, TAPN), constraints (OCL), and test suites, designed with both functional logic and human centred features that reflect real daycare challenges as discussed in the project introduction and are ready for handoff to development.

1.1. Project description

This system provides two primary functions, developed in response to real-world challenges presented in the video materials. Childcare centres depend on stable staff availability to ensure child safety and the smooth running of routines. However, as the video emphasizes, last-minute absences, lack of substitute coverage, and insufficient communication can create serious disruptions.

Firstly, in the Vacation Planning model, the employees log in, submit requests for absence periods, reasons, and documents if needed. This process follows strict rules, such as the doctor's note for sick leave's above 2 days.

Regarding the system, it changes the colour of absence entries based on approval and sends alerts for overlapping absences. Finally, the director reviews and approves or rejects requests, while authorized staff can approve of modified schedules. It is also in this model that the parent (or the teacher) fulfils the emergency care forms, used to support urgent childcare adjustments, during summer vacation.

Additionally, the "How Do I Feel Today?" system relies on children's check-in and records their daily emotional status via a mood selector. This data is than aggregated for directors to monitor their well-being trends and proactively manage staff support or adjust scheduling during stressful periods, as emotional strain and burnout are real concerns in early childhood education environments.

By combining scheduling automation with empathetic children (and staff) monitoring, this system reduces administrative burden, improves resource planning, and creates a healthier, more resilient childcare environment.

1.2. Team and workflow

The project was developed collaboratively under a flexible foundation structure by a multidisciplinary teamwork of six members in the beginning, ending with five members, after the first delivery, since one of them dropped the course. In this way, the presented team is the one that followed the project until the end. The product ownership was shared among all members ensuring a common understanding of the goals and scope. However, the Scrum Master role rotated between sprints 1 to 4 having been played by Lara Gürbüz, Matys Tavoso, Rodrigo Coelho and Rita Belo.

The team organized itself into two subgroups to work in parallel on different models in the project (one for the Vacation planning and the other about the topic "How I feel today?") and their respective deliveries, such as diagrams, documentation, and test development. Despite the division, all major modelling decisions and validations were carried out collaboratively in team sessions to ensure consistency across artifacts.

We followed a sprint-based workflow, holding work meetings along each sprint phase, for that we used collaborative tools to ensure everyone could contribute simultaneously and transparently. Our testing and documentation processes were supported by group reviews and AI-assisted drafting sessions, which allowed the team to co-create/refine content efficiently.

1.3. Tools used

As mentioned previously, to support the modelling, testing and documentation of the system, the team made use of a wide variety of tools,

selected for collaborative software engineering work. These tools and their main goal are described in the following:

- **Lucidchart:** for all de UML diagrams (as use case, class, sequence and communication), allowing us to work in cooperation in each one-off them;
- **TAPAAL:** for the Timed Arc Petri Nets (TAPN), modelling and validation;
- **OpenAI ChatGPT3.5 (GPT4-o) and Gemini 2.5 Flash:** were used as an AI-assisted for the test case generation;
- **GitLab:** version control and markup documentation along the semester;
- **UI Concept:** mood selector (happy/sad icons) visible each morning on entrance dashboard, stored as "*moodLog*" per date.

Together, they enabled a fluid and efficient development process, even when team members worked asynchronously or across different roles.

2. Vacation Planning

2.1. Problem statement/definition

By analysing the video interview with the director, we identified key challenges and difficulties in the current vacation planning process. During the interview, the director pointed out several gaps and weaknesses in the system, as well as opportunities for improvement. This insight formed the basis for our understanding and guided the development of our proposal.

2.1.1. Main problem statement

The main issue identified at the childcare centre was the inefficiency of the existing vacation planning process. Being entirely manual and paper-based, it not only consumed time but also created unnecessary complexity. The responsibility was fully centralized in the director, which contributed to feelings of overwhelm and increased the probability of errors.

These challenges can be summarized as follows:

- Manual and paper-based workflow: The entire process relied on physical forms, making it slow and inefficient;
- Director overload: All responsibilities were centralized in the director, significantly increasing her workload;
- High risk of errors: Manual data handling raised the likelihood of mistakes and miscommunication;
- Lack of coordination: Staff members had limited insight into each other's schedules, complicating the planning of adequate team coverage.

2.1.2. Sub-problems

In addition to the broader challenges of a manual and centralized process, several sub-problems were identified that further contributed to the system's inefficiency.

The key sub-problems identified were:

- Lack of automatic conflict detection: Overlapping vacation requests had to be managed manually, increasing the risk of scheduling gaps.
- No real-time updates: Changes to submitted requests were difficult to track and often required restarting parts of the process.

- Unstructured approval workflow: There was no formalized method for submitting, reviewing, and confirming requests, leading to inconsistent communication;
- Weak historical tracking: Past vacation data wasn't stored in a usable format, limiting its value for future planning or accountability.

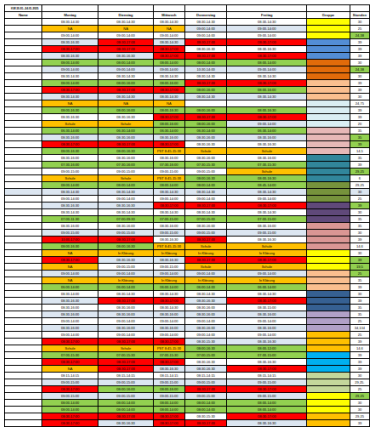

Figure 1- Dienstplanung before

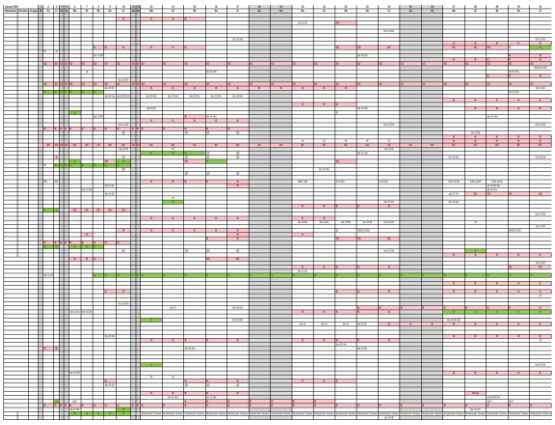

Figure 2 - Anwesenheitsliste, before

2.1.3. Glossary of Terms

With the purpose of providing a quick and clear list of definitions for better reading and support while analysing the project (diagrams and technical artifacts), this glossary collects the key actors, concepts, variables, and system functions present in the Vacation Planning module.

2.1.3.1. Actors

Term	Description
Childcare Worker	Main user of the system. Submits, views, cancels, and modifies absence requests.
Teacher	Same permissions as a worker, plus submits emergency care forms.
Director	Approves/rejects absence requests, receives alerts, views and filters requests.
Authorized Personnel	Modifies the vacation calendar in case of unexpected changes.
System	Enforces rules, processes alert, validates requests, and updates the calendar.

Table 11 - Actors for Vacation Planning

2.1.3.2. Main Classes and Domain Concepts

Concept/Class	Description
AbsenceRequest	Object storing a single vacation/sick leave submission.
VacationCalendar	Central class storing all staff absences and managing calendar visualization.

AbsenceType	Encodes reason for leave (e.g., Vacation, Sick) and related documentation.
RequestStatus	The state of an absence request: Pending, Approved, Rejected, or Cancelled.
Alert	Object generated automatically when business rules are violated (overlap, update).
EmergencyCareForm	Form submitted when fast schedule rearrangement is needed.
User (Superclass)	Abstract class from which Teacher, Worker, Director, etc. inherit.

Table 22 - Main classes and Domain Concepts for Vacation Planning

2.1.3.3. Key Variables & Attributes

Variable	Type	Description
startDate	Date	First day of the absence request.
endDate	Date	Last day of the absence request.
Reason	String	Reason for absence ("Vacation", "K" for sick leave).
Status	String	Status of the request.
hasDoctorNote	Boolean	Indicates whether a medical certificate is provided.
maxRequestsPerDay	Integer	Maximum number of allowed simultaneous absences.
vacationDaysLeft	Integer	Number of vacation days available to the user.
Role	String	User role: Childcare Worker, Teacher, Director, etc.

Table 33 - Key Variables & Attributes for Vacation Planning

2.1.3.4. System Functions & Use Case Actions

Function / Method	Description
submitRequest()	Sends an absence request to the system.
ApproveRequest()	Director approves a pending request.
rejectRequest()	Director rejects a pending request.
cancelRequest()	Worker cancels their own pending request.
modifyRequest()	Worker modifies a submitted request (before approval).
uploadDoctorNote()	Adds a document to a sick leave request (required for >2 days).
sendAlert()	System triggers an alert based on defined conditions (overlapping, update, cancel).
viewCalendar()	User views the shared calendar with color-coded absences.
filterRequests()	Director filters requests by date, department, or role.
exportSummary()	Director exports absence summaries or calendar snapshots.
trackVacationBalance()	System automatically calculates days taken and days left.

Table 4 - System Functions & Use Case Actions for Vacation Planning

2.2. User stories analysis

To ensure a user centred design, we structured system functionalities around user stories. Each story describes a goal from the user's perspective and is categorised by epic, priority, and mapped to related use cases. In the table below is a selection of user stories extracted from our stakeholder analysis, which were fundamental in shaping the use case diagram, defining class responsibilities, and guiding the test suit design, besides defining a base for the project.

Nr	User story	Priority	Epic	Feature (Use Case)
1	As a childcare worker I want to be able to request/inform when I will be absent so that everybody knows I won't be at my Workplace.	1	yes	Submit absence request
2	As a childcare worker I want to be able to choose the reason of my absence so that I can be in line with the daycare centre regulation.	1	no	Select absence reason
3	As a childcare worker I want to be able to provide the documents justifying my absence so that I can proof my absence.	1	no	Upload absence documents
4	As a childcare worker I want to be able to see the schedule of my absence so that I can keep in track of my previous days.	3	no	View calendar
5	As a childcare worker I want to see the status of my vacation requests (approved or pending), so I know if my leave is confirmed.	2	no	View request Status
6	As a childcare worker I want to see how many vacation days I have left, so I can plan my time off accordingly.	3	no	View remaining absence days
7	As a childcare worker I want to view the team vacation calendar, so I can avoid overlapping my leave with colleagues.	2	no	View team calendar
8	As a childcare worker I want to update or cancel an absence request, so I can respond to changes in my personal situation.	1	yes	Update request
9	As a childcare worker I want to be able to login so that I can access the platform with the right privileges and dispose my personal account.	1	no	Login
10	As a teacher I want to have access to the emergency-care form so that I can give it to the parents to fill it out.	1	no	Access Emergency-Care Form

11	As the director I want to view all submitted vacation requests from staff (with detailed info), so I can review and manage them efficiently.	1	no	View submitted requests
12	As the director I want to approve or reject vacation requests, so I can ensure there is enough staff available.	1	no	Approve/Reject requests
13	As the director I want to receive alerts when too many staff members request the same period off, so I can intervene early.	2	no	Receive alerts
14	As the director I want to receive alerts when a staff member changes their request, so I can be aware of it.	2	no	Receive alerts
15	As the director I want to see the number of vacation days each employee has taken or still has left, so I can monitor fairness and compliance.	2	no	Monitor absence day balance
16	As the director, I want to filter vacation requests by role or group (e.g., preschool, nursery), so I can balance leave within each department.	2	yes	Filter request
17	As the director, I want to export vacation plans or summaries for documentation or internal reporting purposes.	3	yes	Export plans/summaries
18	As an authorised personal I want to be able to modify the planning if an update to a request occurs so that the planning is up to date.	1	yes	Modify calendar
19	As a system I want to track the number of vacation days each employee has taken or still has left, so that I can keep it up to date.	1	no	Track absence balances automatically
20	As a system I want to generate the number of children, childcare times, allocation of groups needed during the summer holidays so that the staff can easily organize themself.	2	yes	Generate summer plan
21	As a system I want to underline present and absent personal so that the information is up to date and clear.	2	no	Highlight absences
22	As a system I want to use different colours when a correction of the planning is done so that everyone can see the last changes.	3	no	Colour-code planning
23	As a system I want to be able to highlight the reason of absence accordingly so that it fulfils the needs of the director.	3	no	Colour-code planning

Table 5 - User Stories for Vacation Planning

2.2.1. Use Cases Analysis

Based on the user stories and stakeholder needs, we defined a set of formal use cases. Each use case describes a specific interaction between an actor and the system, and is used to derive system behaviour, class responsibilities, and testing logic. Although already shown in the table from the previous chapter, connecting them with the user stories, here we can observe how they are connected to actors.

Nr	Feature (Use Case)	Actor	Description
1	Submit absence request	Childcare Worker	User submits a vacation or sick leave request with dates and reason.
2	Select absence reason	Childcare Worker	User selects a standardized reason (vacation, sick, personal).
3	Upload absence documents	Childcare Worker	User uploads supporting documents (as doctor's note for sick leave > 2 days).
4	View calendar	All users	User views their own personal calendar with approved and pending absences.
5	View request Status	Childcare Worker	User checks whether their request is pending, approved, or rejected.
6	View remaining absence days	Childcare Worker	Displays how many vacation days are left for each user.
7	Track absence balances automatically	System	System monitors each user's vacation usage automatically.
8	View team calendar	Childcare Worker	Allows viewing of all staff absences to avoid overlaps.
9	Update request	Childcare Worker	User modifies the dates or reason of a pending request.
10	Login	All users	System authenticates user and grants access according to role.
11	Access Emergency-Care Form	Teacher	Teacher accesses and fills out emergency-care planning form.
12	View submitted requests	Director	Director views all submitted absence requests with full details.
13	Approve/Reject requests	Director	Director reviews and decides on requests to ensure proper staffing.
14	Receive alerts (overlap, modifications)	Director	Director is notified when a request is updated/cancelled; or if multiple absences overlap during same period.
15	Monitor absence day balance	Director	Allows directors to track each employee's used and remaining days.

16	Filter request	Director	Enables filtering by department, role, or date range.
17	Export plans/summaries	Director	Exports calendar summaries or staff reports for documentation.
18	Modify calendar	Authorized Personnel	Authorized staff adjusts calendar entries if sudden changes occur.
19	Generate summer plan	System	Creates a summary of group needs and availability during summer break.
20	Highlight absences	System	Marks approved absences visually in the shared calendar.
21	Colour-code planning (approved, pending)	System	Approved absences shown in green and pending absences shown in grey.

Table 6 - Use Cases for Vacation Planning

Table 6 details the core use cases identified through our stakeholder analysis and user stories. Each row describes a functional interaction between the system and a specific actor, forming the basis for our behaviour modelling (UML, EFSM, TAPN) and testing strategy.

2.2.2. Requirements

The system is defined by both functional and non-functional requirements. Functional requirements define what the system does, while non-functional requirements define how well it performs under constraints, describing usability, performance, and quality of attributes.

Functional Requirements

- The system shall allow users to log in based on their role.
- Childcare workers shall be able to submit absence requests with required fields.
- The system shall validate input data, including date ranges and absence reasons.
- The system shall enforce rules (based on business rules listed).
- Directors shall be able to approve or reject requests.
- Alerts shall be generated when there are too many absent requests for the same period.
- Authorized staff shall be able to modify or cancel existing plans.
- The calendar shall update in real time and display request status using colour codes.
- The system shall log emotional trends and present summaries to teachers.

Non-Functional Requirements

- The system shall be accessible through desktop and tablet devices.
- Interface shall be intuitive for non-technical users (workers, directors).
- System shall support multiple concurrent users without conflict.

2.3. System design and architecture

In this section, it is described the technical architecture and structural design of the Vacation Planning module. It covers the system components, their interactions, and how business logic is implemented and enforced across models.

2.3.1. Use Case Diagram

The UML Use Case Diagram models main interactions between different user roles and system for a Vacation Planning of a daycare staff management platform. It captures:

- Functionalities available to each Childcare Worker, Teacher, Director, Authorized Personnel, and the System itself;
- It was chosen to combine some actors using hierarchy to diagram simplification, being easier to read and implement (in terms of processing time). These roles are associated to each childcare profile and become clear once they login in the platform;
- The main system shows as an actor to process independently (and visibly in diagram) each submitted request, send/receive alerts, calendar management and general automatic behaviours.

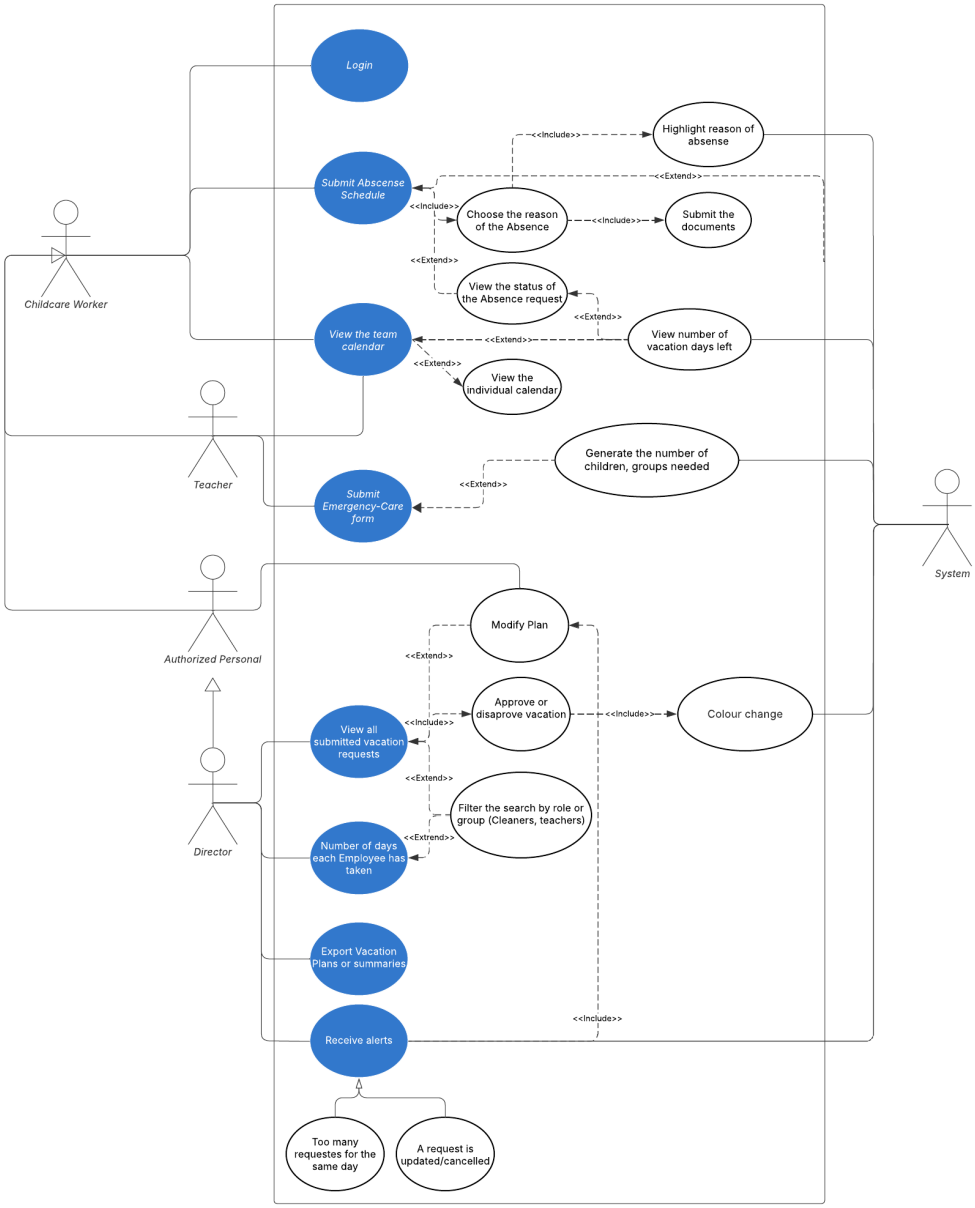

Figure 3 - UML Use Case Diagram for Vacation Planning

2.3.2. Class Diagram with OCL

This UML Class Diagram represents the full structure of the Vacation Planning subsystem in a daycare application, and it captures:

- All main classes and their attributes/methods;
- The embedded logic using Object Constraint Logic (OCL) to enforce real-world rules;
- Ensures that the system behaves correctly, consistently, and transparently, especially when handling absences, approvals and emergency care;
- The separation of the user roles, which clarify each one's rights and obligations;

➢ The separation of the system connected roles present a clean view of conditions and tasks;

➢ the OCL conditions were based on the reasons and conditions information.

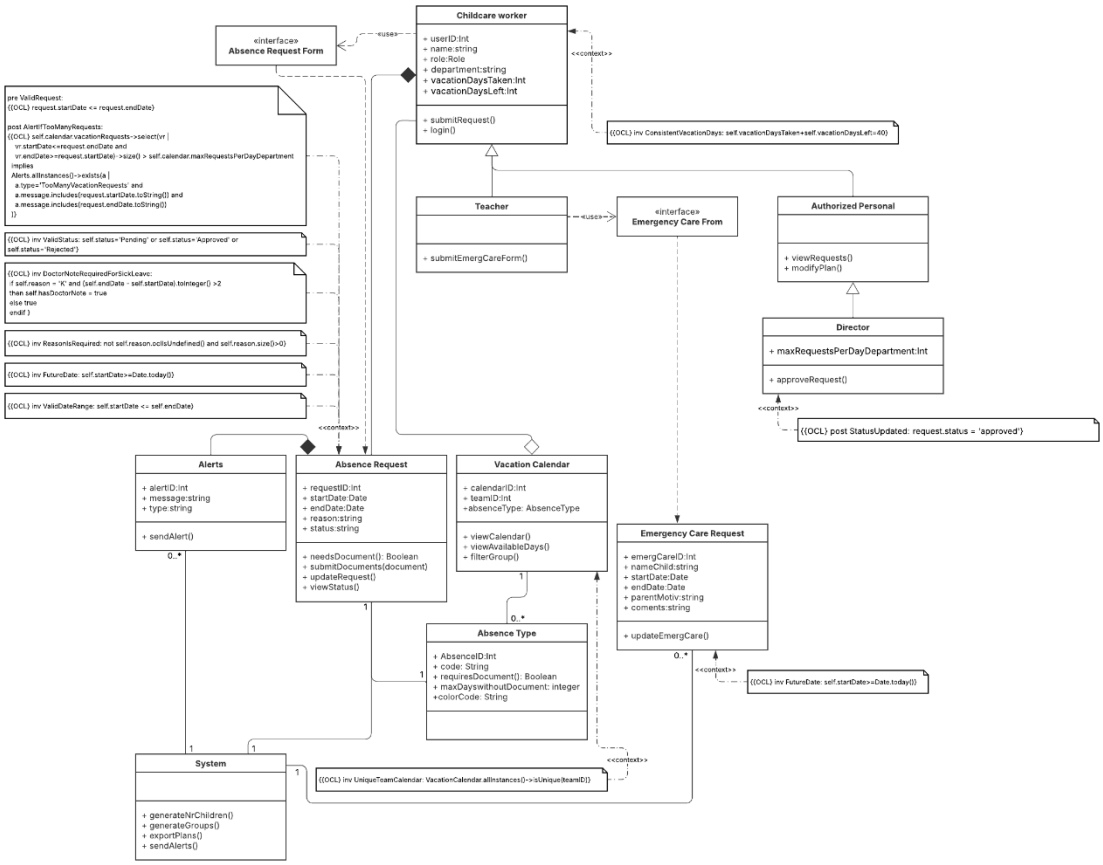

Figure 4 - UML Class Diagram with OCL for Vacation Planning

Other notes:

➢ The request needs to be accepted/rejected by the director, once she asked to have control over the requests;

➢ The use of composition (*AbsenceType* inside *VacationCalendar*) enforces lifecycle ownership;

➢ It was important for us to include but in a "parallel" system both *AbsenceRequestForm* and *EmergencyCareForm*, to clarify UI-system boundaries.

2.3.3. EFSM (Extended Finite System Machine) Diagram

The diagram models the dynamic behaviour of the system changes over time, and how the Requests are processed internally based on conditions and inputs. In this order of ideas, the diagram concerns:

➢ The full flow from login to vacation request processing, concerning access control (role validation)

➢ The request status management (pending, modified, accepted, rejected) and associated system-triggered actions as alerts and calendar updates.

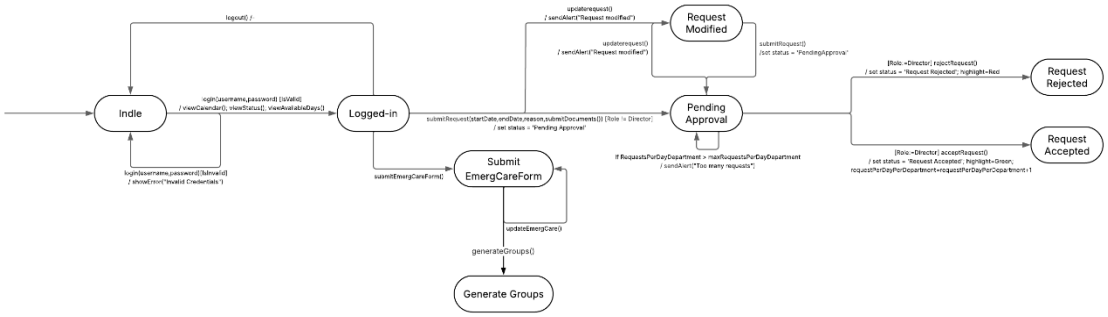

Figure 5 - EFSM Diagram for Vacation planning

Other notes:

➢ It showed of importance distinguish the Role equal/different form "Director" to prevent unauthorized access to approval steps (essential to security and control).

➢ As requested by the director, for each request modification, cancelation or if there is too many for one specific day in the same department, the system sends an alert.

➢ The loops are essential, so the system does not reach a "dead end" ramification and allow the user to keep on using the software.

2.3.4. Activity Diagram

The activity diagram captures the step-by-step behaviour of users during a typical use of the platform, from login to vacation request resolution. It focuses on:

➢ User/system responsibilities by using swimlanes for each role, possible by the login (these credentials are associated to a certain role);

➢ Interactions between roles, which express easily the delays between actions because of other roles/swimlanes tasks;

➢ Each decision point led to real-life decision paths based on, for example, the absence justification.

Other notes:

➢ As requested by director, once pending, if the request is accepted, this can be highlighted by colour changing in calendar;

➢ After the main page, there are two different systems (*AbsenceRequest* and *EmergencyCare*) working in parallel.

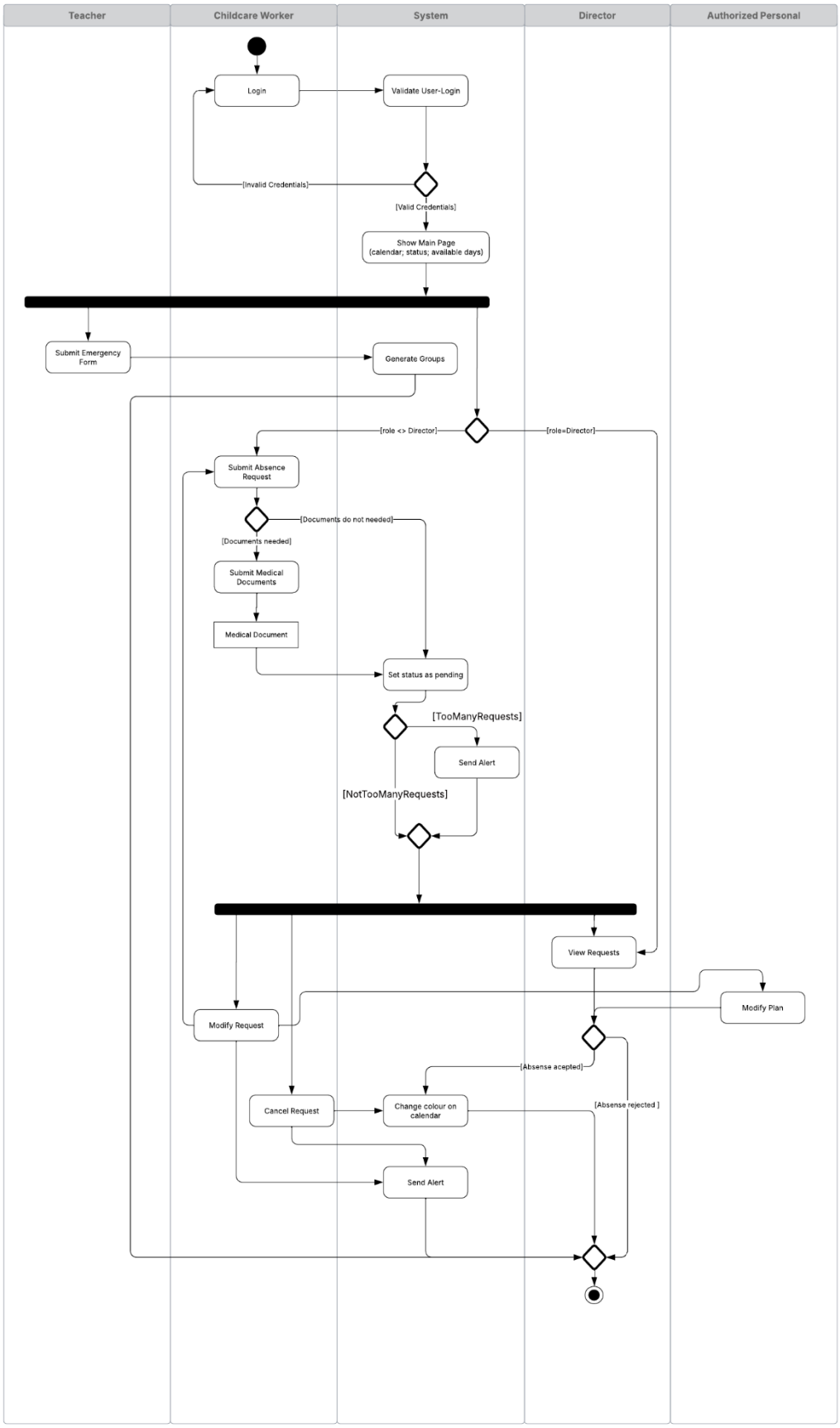

Figure 6 - Activity Diagram for Vacation Planning

2.3.5. Sequence Diagram

This UML Sequence Diagram models the chronological flow of interactions between different system participants (users and system) and illustrates:

- How/when users interact with the system and its respond based on conditions (as overlap days or modifications);
- Message exchange order and roles involved at each step highlighting normal flow and alternative paths (too many requests or request modification/cancellation).

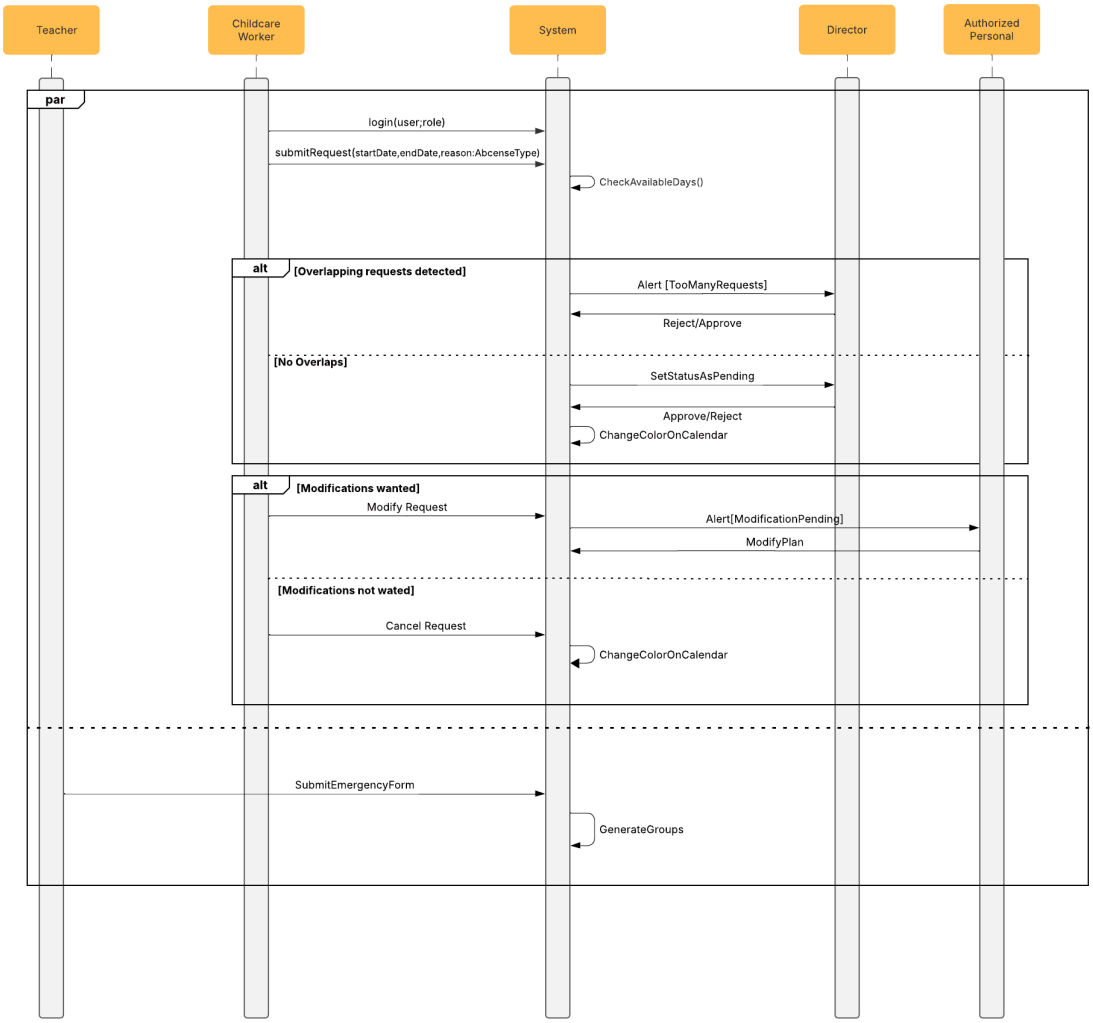

Figure 7 - Sequence Diagram for Vacation Planning

2.3.6. Communication Diagram

The diagram focuses on how roles interact with each other, highlighting messages exchanges between each other, but without emphasizing the timing of each action. In core, this diagram complements the sequence diagram by highlighting the structural relationships and message flow paths between system components. Some key features:

- all communications are numbered to show order of action, maintaining the logic sequence of the system, without relying on vertical timelines;
- whenever it is needed there are bi-directional message flows to show how authority and responsibility move back and forth between users and system elements.

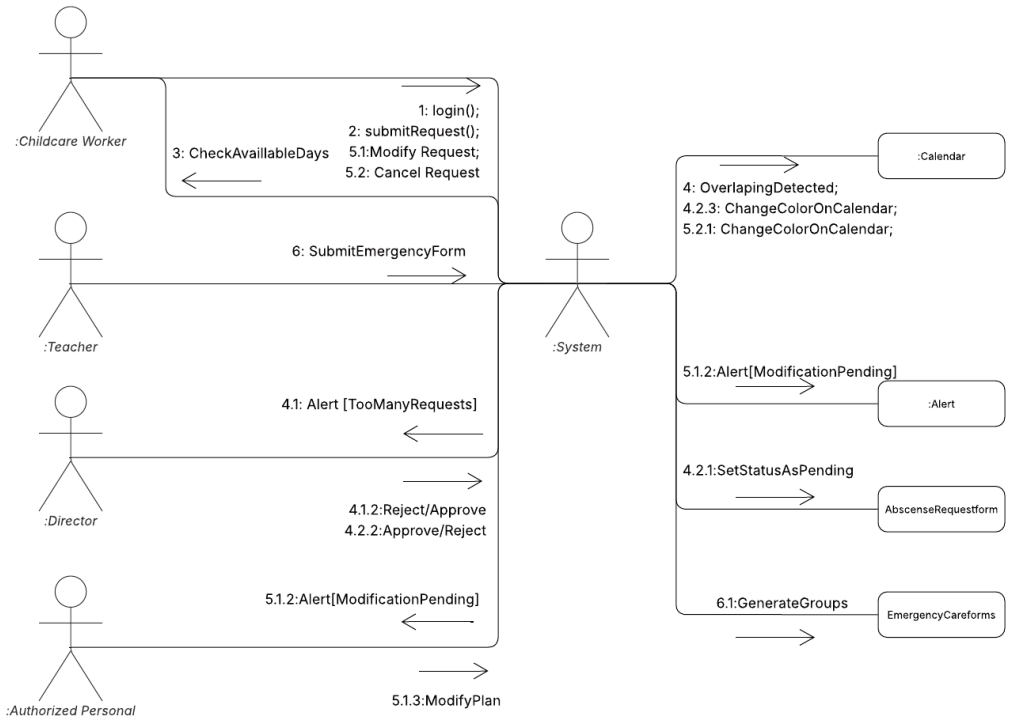

Figure 8 - Communication Diagram for Vacation Planning

2.3.7. TAPN (Timed-Arc Petri Net) Diagram

Petri Nets represent the different possible states of the system using tokens, transitions and time constraints. Besides formalizing how the system behaves in terms of process state, event timing and resource usage, it captures:

- Submission and validation of absence requests with conditional logic for documentation (sick leave over 2 days)
- Pending approvals and director decisions connected to alerts (for request overload, etc)
- Cancellation/modification paths
- The *Places* correspond to the system states (help model conditions/transitions between phases)
- The *Transitions* represent actions/events (to which may be associated timeouts or conditions)
- the *Time constraints* reenforce realistic deadlines and keep the process accountable, written in *hours*

Other notes:

- The time intervals were based on common sense, so they can be changed accordingly to the director's wishes:
  - [0,0] show that the transition is immediate as login and form submissions;
  - [0,1] show that the transition must happen within 1h as the alerts;
  - [0,2] show that the transition must happen within 2h as setting the pending status;
  - [0,48] show that the transition must happen within 48h (2 working days) as the cancelation of an absence request;
  - [0,72] show that the transition must happen within 72h (3 working days) as the modification of an absence request;
  - [0,120] show that the transition must happen within 120h (5 working days) as accepting/rejecting an absence request.

Bureaucratic notes:

- These dates do not correspond with uploads done on GitLab;
- This diagram was fulfilled by the same members as the one above;
- For a clearer view, there is the same representation of this diagram using Lucidchart as the others, however, the diagram was created using TAPAAL as seen in the figure bellow.

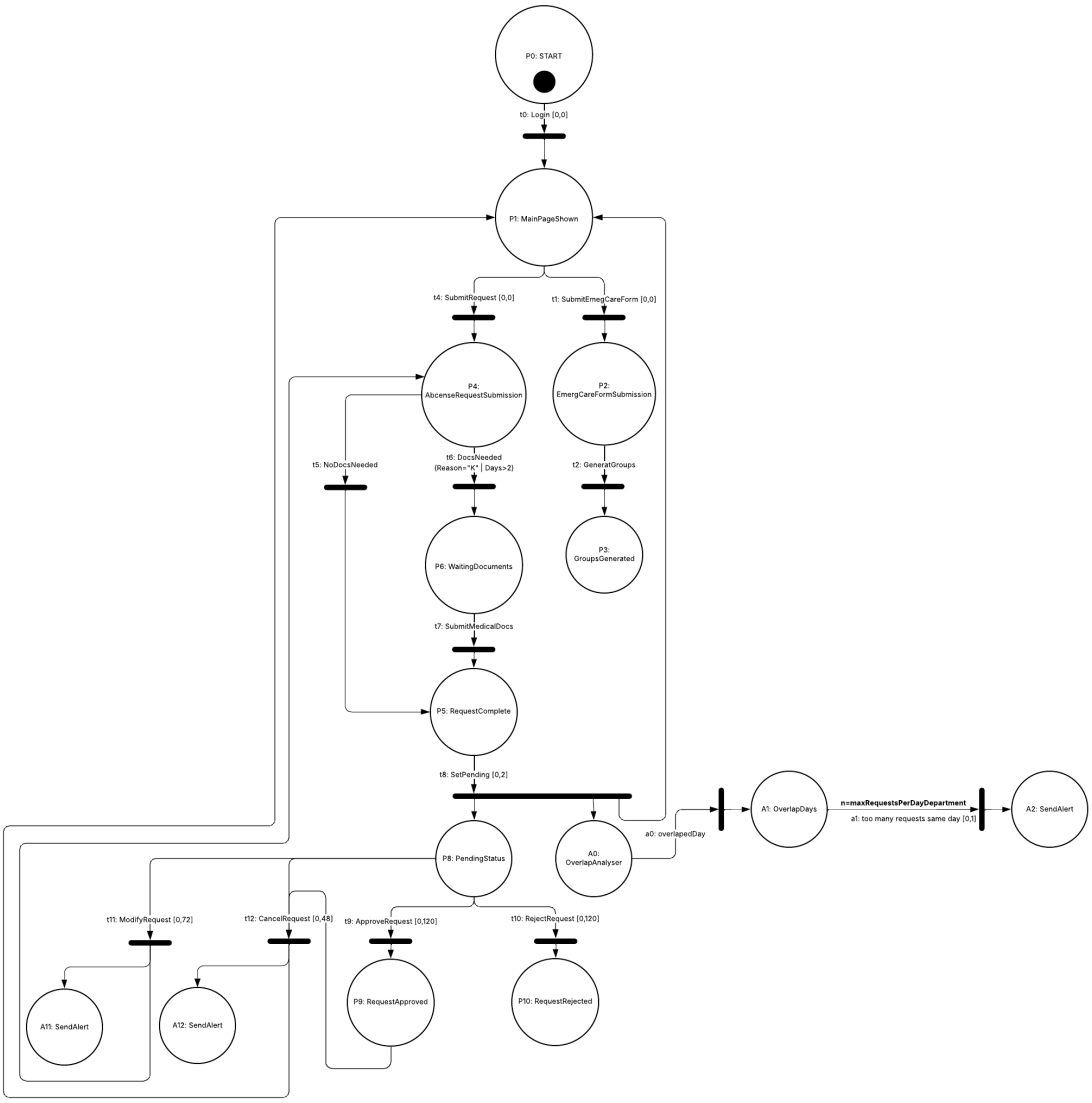

Figure 9 - Clearer view from TAPN for Vacation Planning

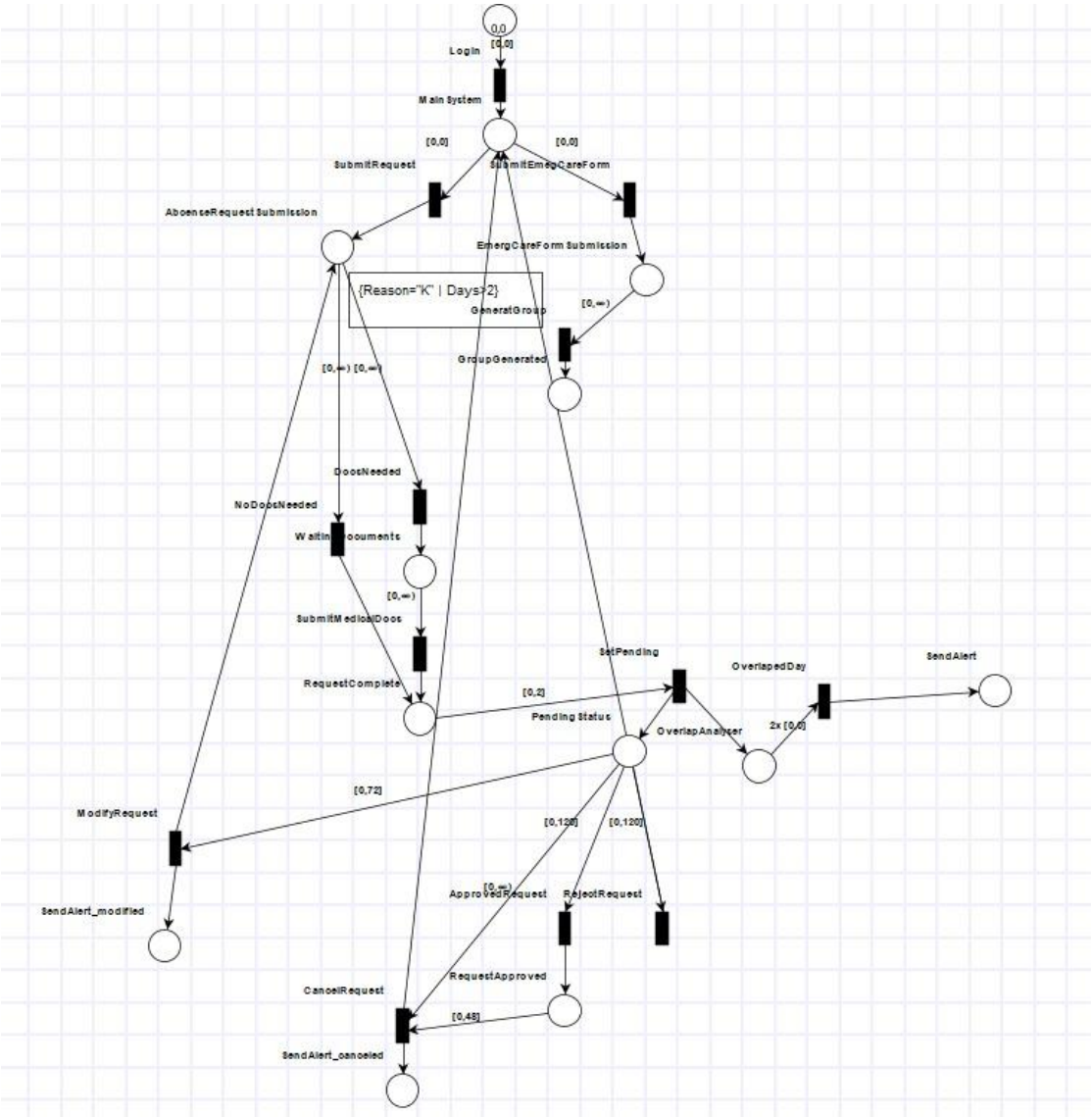

Figure 10 - TAPN Diagram for Vacation planning, using TAPAAL

2.3.8. Architecture

The chosen approach was to represent our system architecture using a layered approach, as it offers a clear and structured separation of concerns. This model fits well with the scale and requirements of our application and ensures a logical hierarchy between components.

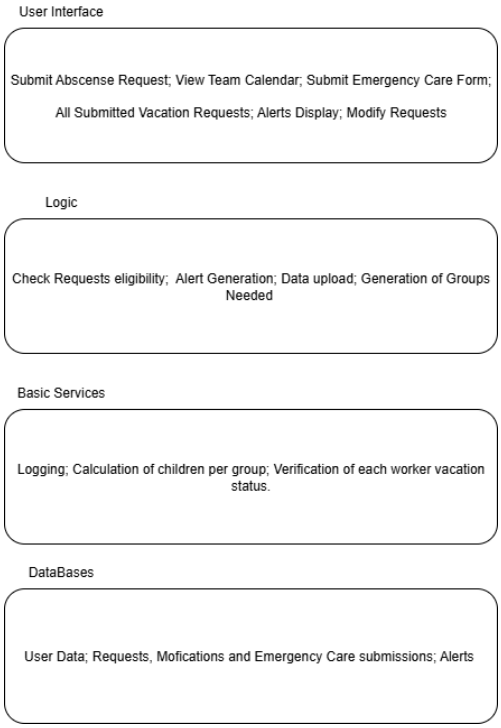

Figure 11 - Architecture for Vacation Planning

The **User interface** layer is responsible for all user interactions. It includes functionalities such as submitting absence or emergency care requests, viewing the team calendar, accessing all submitted vacation requests, receiving alerts, and modifying existing requests.

The **Logic** layer handles the core processing rules of the system. It checks the eligibility of submitted requests, generates necessary alerts, manages data uploads, and determines the grouping of children based on staff availability.

The **Basic services** layer supports the logic layer by offering essential operations, including system logging, calculating the number of children per group, and verifying the vacation status of each worker to ensure proper planning.

Lastly, the **Databases** layer stores all critical data needed by the system to function. This includes user information, submitted requests, emergency care forms, modifications, and alerts.

2.4. Testing Approach/Strategy (test-suites)

For testing, *Gemini 2.5 Flash* was used, a large language model, to generate and refine black-box test cases based on our user stories, activity diagram, and constraints modelled in OCL and EFSM. Since the project is model-based and not yet implemented in code, all tests were performed as logic-based simulations. This means test cases were interpreted manually against the

system's behavioural models and class rules to verify alignment and expected flow.

For the validation of workflows, the technique found most suitable for validating workflows within our domain was the User Story Testing (a black-box approach). FSM-based testing was also considered, but the clear user-centred nature of our requirements made story driven testing more accurate and understandable.

For that, we started by prioritising the user-stories. Then, Gemini AI was used generate 2 test cases for priority one user stories and individually analysed them if they were accordingly to the project requirements. The suite cases are shown in the table presented bellow organised including a simple explanation of the test and what it is expected as a result.

User Story	Test Case ID	Test Case Description	Type of Test	Steps	Expected Result
User Story 1: As a childcare worker I want to be able to request/inform when I will be absent so that everybody knows I won't be at my Workplace.	TC 1.1	Submit New Absence Request (Valid Data)	Positive	1. Childcare worker logs in. 2. Navigate to the "Submit Absence Request" feature. 3. Enter valid start and end dates for absence. 4. Select a valid reason for absence. 5. Click "Submit" or "Request Absence".	System confirms successful submission. Request status is "pending". Director can view the request.
	TC 1.2	Submit New Absence Request (Missing Required Fields)	Negative	1. Childcare worker logs in. 2. Navigate to the "Submit Absence Request" feature. 3. Attempt to submit the form with one or more required fields left blank (no start date).	System displays an error message indicating missing required fields and does not submit the request.
User Story 8: As a childcare worker I want to update or cancel an absence request, so I can respond to changes in my personal situation.	TC 8.1	Update Pending Absence Request (Valid Change)	Positive	1. Childcare worker logs in. 2. Navigate to "View Requests" to see existing requests. 3. Select a pending absence request for modification. 4. Change a detail (end date, reason). 5. Save the changes.	The system updates the absence request. The Director receives an alert about the change.
	TC 8.2	Cancel Pending Absence Request	Positive	1. Childcare worker logs in. 2. Navigate to "View Requests". 3. Select a pending absence request for cancellation. 4. Initiate the cancellation process. 5. Confirm cancellation if prompted.	The system cancels the absence request. The request no longer appears as pending for the childcare worker, and the Director is notified, or the request is removed from their view.
User Story 11: As the director I want to view all submitted vacation requests from staff (with detailed info), so I can review and manage them efficiently.	TC 11.1	View All Pending Requests	Positive	1. Director logs in. 2. Navigate to the "View Requests" section.	A list of all pending absence requests from staff is displayed, including detailed information (staff name, dates, reason, status).
	TC 11.2	View Approved Requests	Positive	1. Director logs in. 2. Navigate to the "View Requests" section. 3. Apply a filter (if available) to show approved requests.	A list of all approved absence requests from staff is displayed, including detailed information.
User Story 12: As the director I want to approve or reject vacation requests, so I can ensure there is enough staff available.	TC 12.1	Approve a Pending Request	Positive	1. Director logs in. 2. Navigate to "View Requests". 3. Select a pending request. 4. Choose the "Approve" option.	The request status changes to "Approved". The childcare worker who submitted the request is notified of the approval.

	TC 12.2	Reject a Pending Request	Positive	1. Director logs in. 2. Navigate to "View Requests". 3. Select a pending request. 4. Choose the "Reject" option.	The request status changes to "Rejected". The childcare worker who submitted the request is notified of the rejection.
User Story 16: As the director, I want to filter vacation requests by role or group (preschool, nursery), so I can balance leave within each department.	TC 16.1	Filter Requests by Role/Group (Preschool)	Positive	1. Director logs in. 2. Navigate to "View Requests". 3. Apply a filter for "Role: Preschool" or "Group: Preschool".	Only absence requests submitted by staff members belonging to the "Preschool" role/group are displayed.
User Story 17: As the director, I want to export vacation plans or summaries for documentation or internal reporting purposes.	TC 17.1	Export All Vacation Plans	Positive	1. Director logs in. 2. Navigate to the export feature ("Export Plans" or "Generate Report"). 3. Select to export all vacation plans. 4. Choose an export format (if options are available, PDF, CSV). 5. Initiate the export.	A file containing the vacation plans or summaries is successfully generated and downloaded or presented for download. The content accurately reflects the current vacation data.
	TC 17.2	Export Filtered Vacation Plans	Positive	1. Director logs in. 2. Apply a filter to the vacation requests (show only "Preschool" requests). 3. Navigate to the export feature. 4. Initiate the export of the filtered data.	A file containing only the filtered vacation plans is successfully generated and downloaded. The content accurately reflects the filtered data.
User Story 18: As an authorised personal I want to be able to modify the planning if an update to a request occurs so that the planning is up to date.	TC 18.1	Modify Planning After Request Update (Approved Request)	Positive	1. An approved request is updated by a childcare worker. 2. Authorized Personnel logs in. 3. Navigate to the planning view (calendar). 4. Locate the entry corresponding to the updated request. 5. Modify the planning to reflect the change.	The Authorized Personnel is able to modify the planning. The updated planning is correctly displayed and saved. The system may highlight the change or prompt for an update to the planning.
	TC 18.2	Modify Planning After Request Approval	Positive	1. A new absence request has just been approved by the Director. 2. Authorized Personnel logs in. 3. Navigate to the planning view. 4. Add the approved absence to the planning.	The Authorized Personnel is able to add the approved absence to the planning. The planning reflects the newly approved absence.

Table 77 - Test Suite using Gemini 2.5 Flash for Vacation Panning

2.4.1. Discussion on the topic

1. ***Were the resulting tests helpful for the refinement of your product artifacts? Assume your project ideas will be developed. Is your AI-generated test suite suitable for testing?***

Yes, as they gave a more precise insight into what and how each user story was being applied in model, through its steps and expected results. Also, helped identifying logical improvements in the EFSM, clarified flow in our activity diagram, and adjusted some class relationships (especially around modification/cancellation of requests). Yes. These tests are detailed, clear, and

can be adapted to automated test scripts in a future implementation. They're based on real user stories and system logic.

2. ***If other students must do the same task for the first time, what advice would you give? What are your lessons learned?***

Even though it takes a bit to define and explain to the AI what you require from it, having to be extra specific on what you want and how you want it, it is a an extremely helpful tool to get things started, but it shouldn't be entirety of the project. As the user we should be critical of its answers and take our time analysing them one by one. Some answers may be good, but many others didn't go entirely according to what we wanted from these tests.

3. ***Will you use again AI tools to generate tests in other contexts?***

In general, they were. Even though some of them were rather repetitive or didn't align with the project, before restricting the test cases to 15 only, after that they all met the requirements expected. Writing a test suite helped validate our diagrams and constraints. It revealed hidden assumptions (for example, when exactly a director is notified). It is also possible to note how essential consistent terminology is between design and test cases.

2.5. Interface design

Although a functional prototype is not currently available, preliminary interface mock-ups have been developed to guide future development.

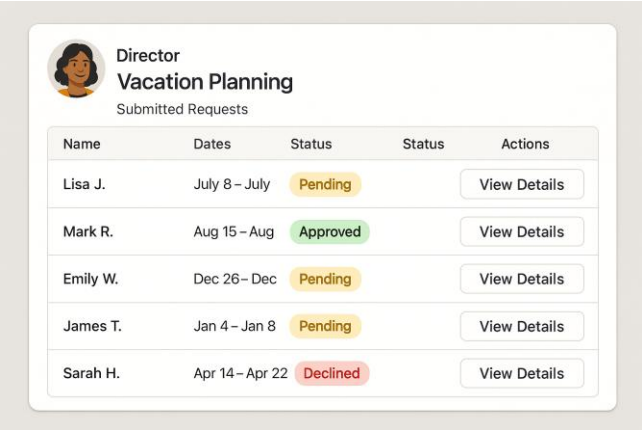

Figure 13 - Director Vacation Request Interface

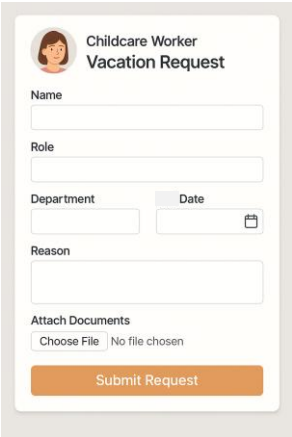

Figure 12 - Vacation Request Interface

These visual concepts aim to illustrate the intended user experience for both directors and childcare workers within the vacation planning system.

3. How do I feel today

3.1. Problem statement

There is a pre-existing approach to the request, which is to ensure that an environment of trust is established for everyone in the center, not only for children but also workers. This involves the choice of emotional cards by the children upon arrival at the center every morning. The next step would be to implement it digitally, in the entrance area.

3.1.1. Main problem

The main goal and idea of the project had already been pre-established by the customer: the opportunity for each child to describe how they are doing with symbols. This would allow feedback on how a single child, a group, or even the whole childcare centre is feeling.

3.1.2. Sub-problems

To achieve this, firstly, the system needs to be able to identify the user: child or adult, which should not share a common interface. Furthermore, it needs to be able to match each entry to a specific child, while implementing a method to aggregate the entries taken individually to a more general output (to the scale of the group or the centre).

3.1.3. Glossary of terms

The actors considered are slightly different from the previous chapter. A user can be either a child, the director, or a worker. The first two are self-explanatory, but here the actor “worker” is slightly different from the vacation planning part. It represents indeed the educator in charge of a group, and not another kind of employee (administrative, kitchen etc.).

3.2. User-stories analysis

ID	User Story	Priority	Epic
1	As a child I want to be able to choose in the morning between different symbols that match my mood, so that I can be more easily understood by adults.	1	No
2	As a child, I want to be able to access the app in the simplest way, so that I can easily use it	1	Yes
3	As a childcare worker I want to be able to consult the mood of each child so that I can discuss with them to know why.	1	Yes
4	As a childcare worker I want to be able to analyze the group’s mood so that I can organize the day accordingly.	2	No

5	As a childcare worker, I want to leave comments when a child selects a mood so that context can be provided to help understand emotional fluctuations.	3	No
6	As a childcare worker, I want to be notified if a child consistently selects negative moods so that I can intervene or discuss the situation with the parents.	3	Yes
7	As a childcare worker, I want to have access to the children mood history over the month so that I can identify issues and take better care of the well-being of the children.	3	no
8	As a director I want to be able to compare the group's answers to see if there are any trends or outliers.	2	Yes
9	As a director I want to be able to get a global overview of the 300 children so that I know the feelings of the children.	1	No
10	As the system I want to be able to identify whose mood belongs to whom so that I can give individualized data analysis.	1	No
11	As the system I want to be able to create statistical boards with the weekly moods so that I can make the analyzing process easier.	2	Yes
12	As a user, I want to be able to select my profile so that I can have personalized access.	1	No
13	As an adult, I want to be able to log into the system so that I can have privileges matching my responsibilities.	1	No

Table 8 8- User Stories for How do I feel Today

3.2.1. Use Cases Analysis

Use Case	Summary	Actors	User Stories (ID)
Choose mood with pictures	Children can choose a visual symbol in the morning to express their mood.	Children	1
Match child with emotion	The app can match a mood selection with the identity of a child for personalized use or analysis.	Children, System	2, 10
View individual child mood	Childcare workers can view individual mood entries, comment on them, and review emotional history.	Childcare workers	3, 5, 7
Receive negative mood alert for children	Childcare workers receive alerts when a child frequently selects negative moods.	Childcare workers	6

View own group mood	Childcare workers can view an overview of the moods within their own group.	Childcare workers	4
Overview over all groups	The system can generate a comprehensive view across all groups to identify trends.	Director, System	8, 9, 11
Select profile	Users can select their profile to access the app in a personalized way.	Users	12
Login	Adults log in to access functionalities aligned with their responsibilities.	Adults	13

Table 9 - Use Cases for How Do I Feel Today

3.2.2. Requirements

The system is defined by both functional and non-functional requirements. Functional requirements define what the system does, while non-functional requirements define how well it performs under constraints, describing usability, performance and quality of attributes.

Functional Requirements

- The system must permit users to log in with their own identification method based on their role.
- Children should be able to select their current mood and their selections must be saved in the system with date and time information.
- Teachers and directors should be able to access the children's mood entries that the system stores together with timestamps.
- Teachers should be able to add comments to children's mood entries and these comments should be visible to relevant users.
- Directors should be able to view all mood data across groups, apply filters and see overall reports and trends.
- The system should send alerts when certain conditions are met, such as repeated negative moods.

Non-Functional Requirements

- The system shall be accessible through desktop and tablet devices.
- Interfaces must be simple, intuitive and suitable for children, teachers, and directors, requiring minimal technical knowledge.
- System shall support multiple concurrent users without conflict.

3.3. System design and architecture

3.3.1. Use cases diagram

The use case diagram documents the functional requirements by representing the actors (childcare worker, user, child, adult, director), secondary actor (system), that we chose to represent explicitly, uses cases (choose his own mood, view individual child mood, login, mood, alerts etc.) and their relationships.

The system distinguishes between two types: child and adult. An adult can either be a worker or the director. That's why in both cases it was used a shared arrow.

From the worker’s point of view, once he is viewing his own group mood, he can decide whether to add a comment or consult the whole history of a child. Since this is optional, the relation between those use cases is represented with an "extend".

On the other hand, there were also considered "include" relationships in the following use cases.

Either from the director’s or the worker’s point of view, the whole child centre (resp. group) requires the system to update the children's score and/or to display them through statistics. Thus, statistics refer to the numerical aspects of the scores, such as computation, average, median, and so on.

Additionally, one major topic of discussion resided on how to easily identify each user, especially the children, since consulting each child’s mood requires indeed the system to be able to match them with their answers to the form. Therefore, the main prerequisite defined is the child choosing his own picture in an album belonging to the group.

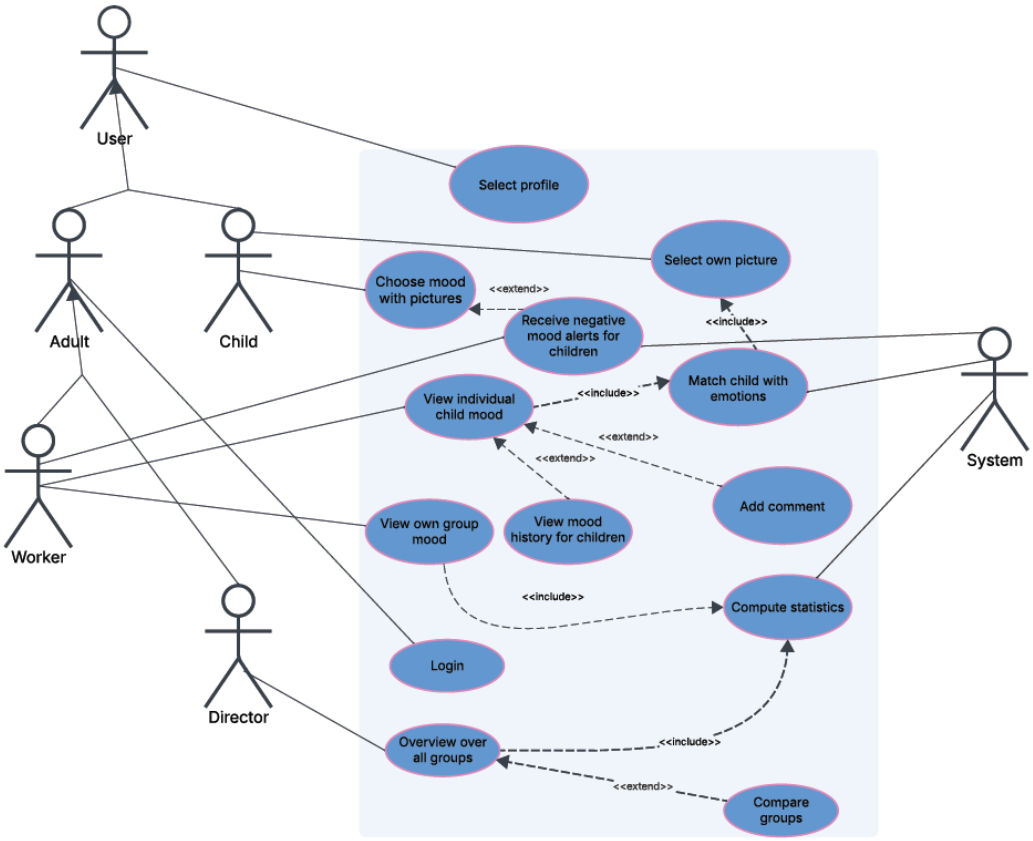

Figure 14 - UML Use Case Diagram for How do I feel Today

3.3.2. Class diagram

There were considered in total 12 classes.

We will dwell first on the class user. It is an abstract class, with the non-abstract method *selectProfile()*. The inheritance allows us to redefine for each kind of user a different way to log into the system. Thus, a user can select his profile (worker, teacher, children), but the login method is different for an adult and for a child.

Regarding the attributes, a group is composed by a certain number of children. Each child has as attributed his own picture, which provide an easy way to connect himself to the system.

Several workers can oversee a same group, with a responsible one.

Each child has his own score, which represents his level of happiness from 0 to 5. Every time the said child fills a form, he has 4 options: a happy smiley, a rainbow, a sad smiley, or an angry smiley. Those are grouped in a *moodSymbol* class, which is for this purpose an enumeration. The first two decrease the score by one, the last two increase it by one.

If the score reaches 5, then the teacher should receive an alert regarding this matter.

On the concern of the relationships between classes, we chose to not explicitly represent the multiplicities other than 0,1 or “*” on the diagram but rather use OCL constraints.

Only one composition is used, between “History” and *MoodEntry*. The history is indeed made of entries which can indeed not "live" outside this class. If the history is deleted, so are the entries. The relationship between Group and Child is modelled as an aggregation. This decision is because a Child can exist independently of any group. It acts as a logical container, organizing children for a specific purpose, without owning or controlling their lifecycle.

Now, we will focus the OCL side. A worker can only be responsible of a group if he oversees the said group.

The constraints allow us also to set up a reliable alert system. Once the score is equal to 5, it can only remain constant or decrease but not increase. We apply the reversed logic for the minimal value, which is 0.

Also, we make sure that an alert is raised when the score is equal to 5, by interacting with the boolean attribute *alertRaised*.

As said above, we used the OCL invariants to define maximal multiplicities. For example, a group can be composed by twenty children or less.

If a child filled the form, then he should be considered as present in the childcare centre.

Finally, the mood entry fill date can only be contained in the interval [start_date, end_date] of the history attributes. Thus, we have necessarily start_date< end_date.

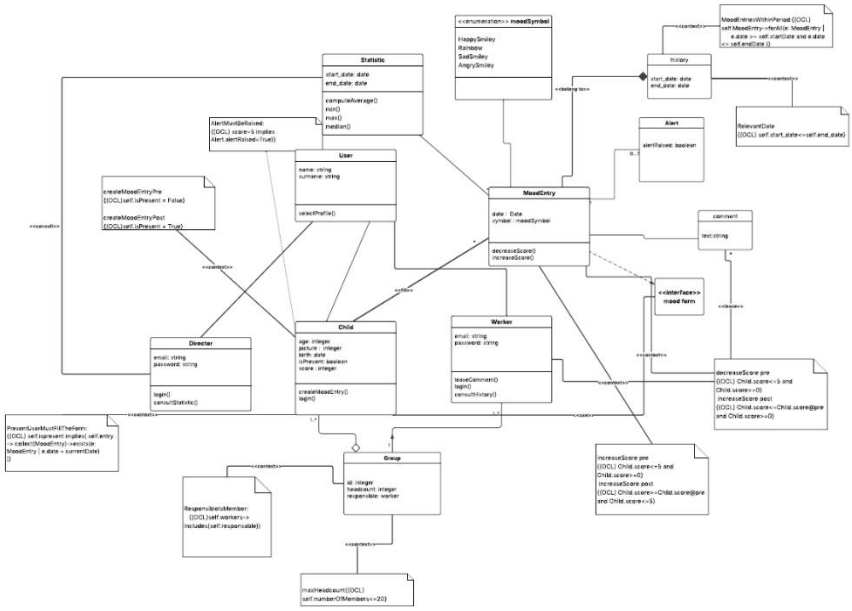

Figure 15 - Use Class Diagram for How do I feel Today

3.3.3. EFSM diagram

The EFSM allows to provide further details about the login method and the computing of the score.

When the system is turned on, it displays the different roles (adult or child). In the EFSM model, adult user roles (teacher, director) are handled via distinct input events (e.g., teacher logged in, director logged in). This design avoids the need for additional internal variables like *userRole*, since the behaviour diverges immediately upon login based on the event.

If child is pressed, then first a group must be chosen. Then, the said child can choose his picture among a roster of children belonging to the group. Once he is identified, he can choose the picture matching his mood of the day. As detailed in the UML class diagram, the form gives the child the possibility to choose between 4 different pictures. This allows more context and details about the feelings involved, but the system sorts them into only two types: sad or happy.

Choosing a happy image decreases his score by one, choosing a sad one increase it. The said score is initialized in the EFSM with an integer “i”, because this number is different for each child (this number is a data stocked, which we can't access with the EFSM). The score being equal to 5 results in an alert being sent. The system displays the next button, which when pressed up allows the next child to complete his mood.

If adult is pressed, the login form is displayed. Following which account is logged in, the user has different privileges. The worker overviews his group's mood, while the director can access the statistics concerning the centre. After 60min without input, the user is logged out.

The EFSM is deterministic, the same input and same predicate leading always to the same transition. All guards are mutually exclusive, guaranteeing predictable system behaviour.

There are no dead states in the EFSM, since all states allow at least one transition under some condition or input. This guarantees that the system can always proceed or return to the main menu and avoids any permanent blocking situation. Such dead state would have been relevant in error-handling, which was not main purpose of this EFSM.

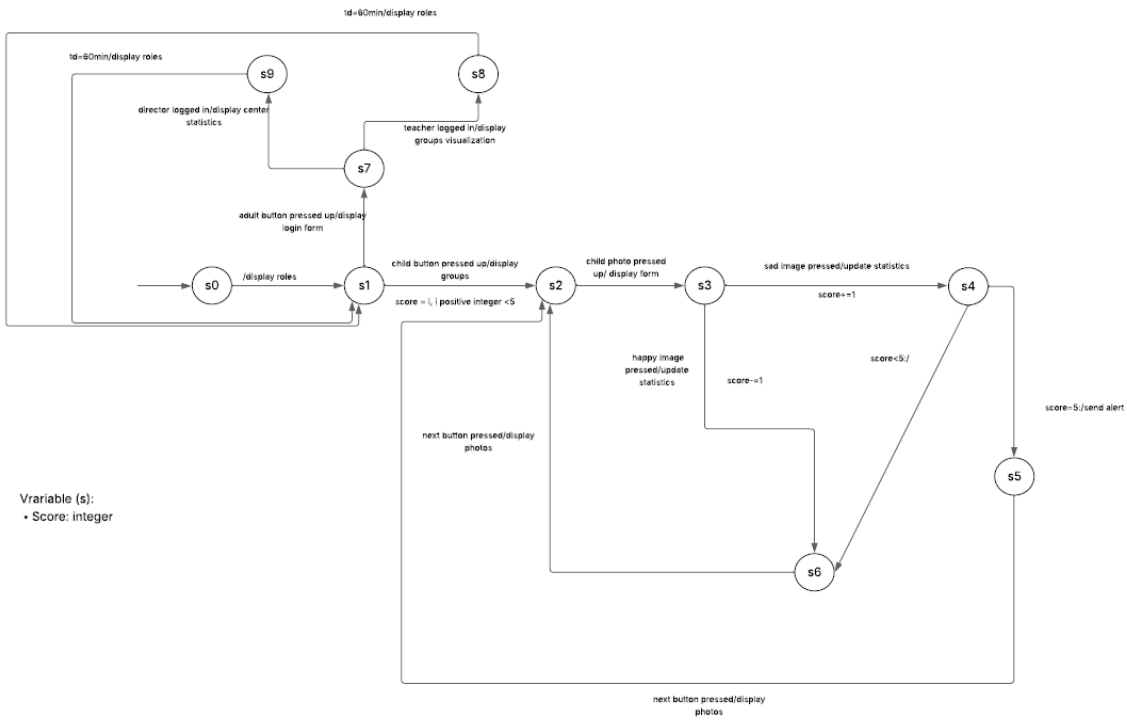

Figure 16 - EFSM diagram for How do I feel Today

3.3.4. Activity diagram

In this diagram, the system is represented as an actor, regarding the role it plays on the logic side. The three other actors are logically the different users, each with its own swim lane.

The activities follow the flow below.

First the child must fill the mood form: it's the initial activity. Then, if he feels bad, and the computed score is superior to the defined level, an alert is sent to the worker in charge. That is the purpose of the decision node with the guard [*isRepeatedNegative*].

Whether an alert is sent or not (synchronization bar), the worker overviews the form filled by the child and the group and may leave a comment if whished.

The final activity lays on the director which has an overview of the whole childcare centre.

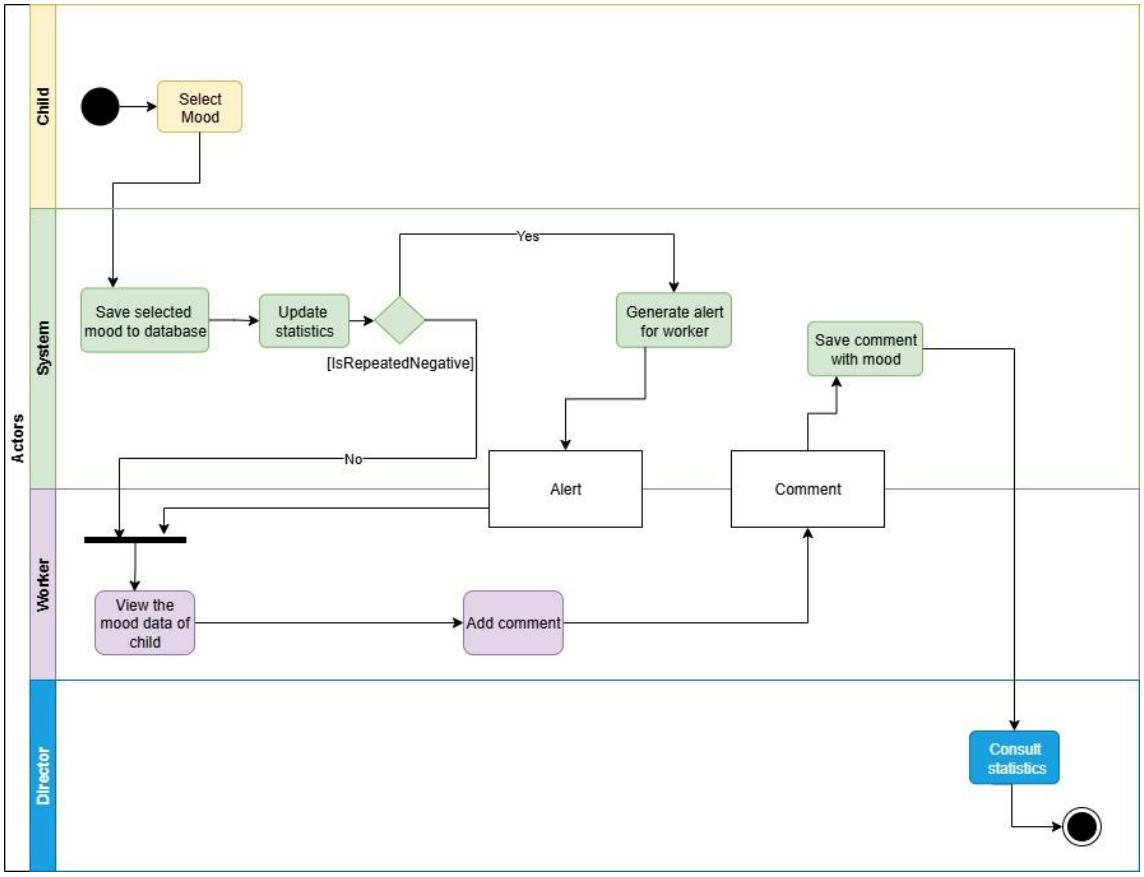

Figure 17 - Activity diagram for How do I feel Today

3.3.5. Sequence diagram

This sequence diagram models the step-by-step interactions between actors. Each participant has a dedicated lifeline showing their specific messages and responses. Conditional behaviour, such as triggering a teacher alert when repeated negative moods are detected, is expressed through branching in the message flow.

After processing one child, the system resets to allow seamless handling of the next. Optional actions like teachers adding comments or consulting mood history are marked with optional message flows. The director's role involves retrieving overall statistics.

Overall, the diagram captures not only the order of events but also how decisions, user roles, and interactions are visually structured and logically handled.

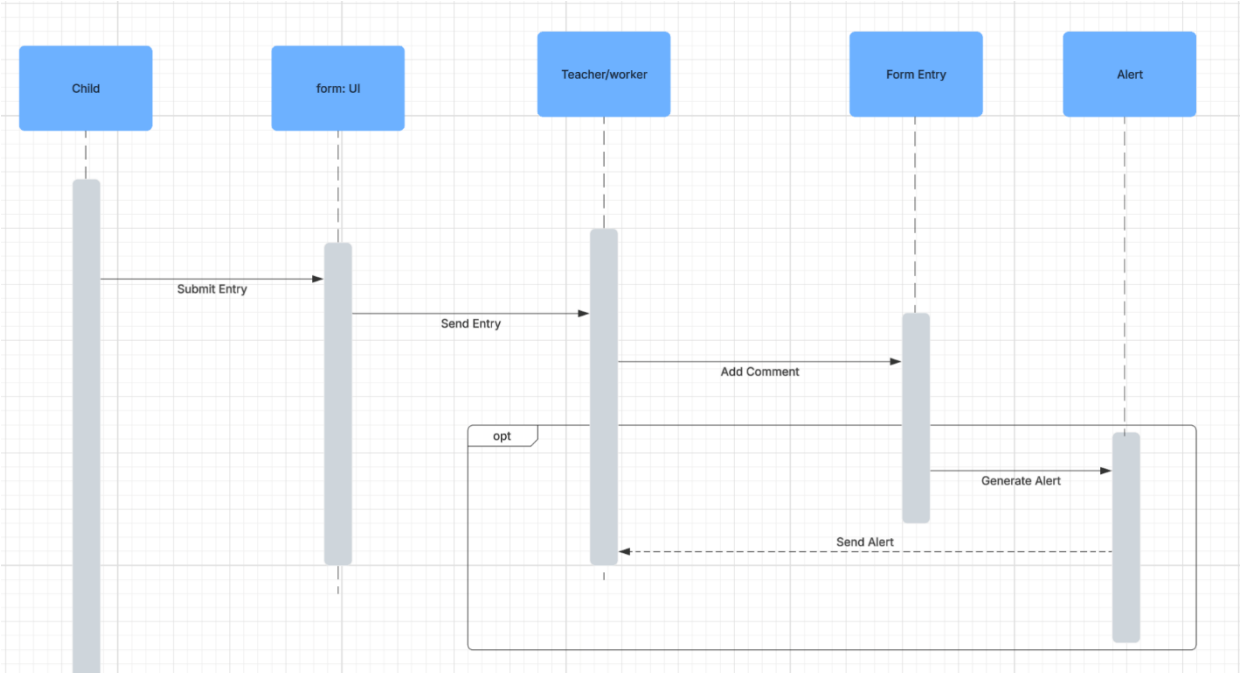

Figure 18 - Sequence diagram for How do I feel Today

3.3.6. Communication diagram

This communication diagram shows how different parts of the "How Do I Feel Today?" system talk to each other. First, the child submits a mood entry to the form UI. Then, the form sends this entry to the teacher or worker, which can add a comment, and is sent to the form entry system. If the mood is bad or repeated too often, the system generates an alert and sends it to the teacher or worker. The numbers on the arrows (1, 1.1, 1.1.1, etc.) show the order of each action.

This diagram helps us understand which part of the system does what, and in what order. It focuses more on the connections between parts, not the exact timing. It's a simple way to see how the child, the teacher/worker, and the system work together.

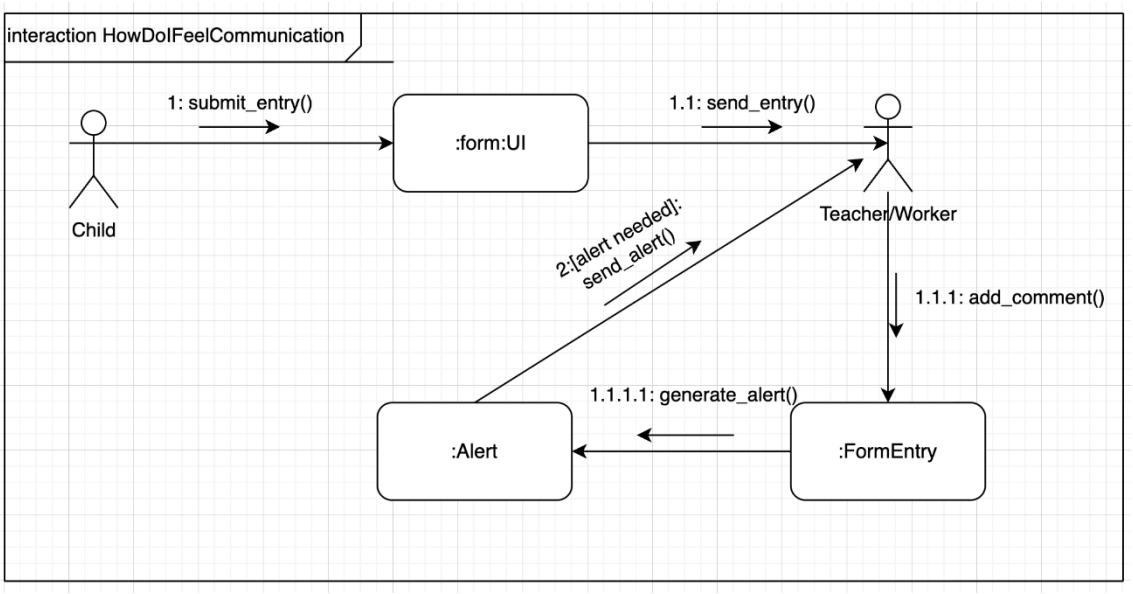

Figure 19 - Communication diagram for How do I feel Today

3.3.7. TAPN diagram

Since the petri nets are a powerful tool to model a distributed system, it allows us to represent several user sessions in parallel.

Thus, a user can select his profile and proceed to the choice of his mood as described above if he is a child. That should not prevent an adult to access his personal account, to view another child data or consult statistics of a previous week for example. A pattern concurrency allows it.

Since different sessions can be opened at the same time, the system must ensure that clashes are impossible with inhibitor arcs. For instance, a teacher can add a comment, but only if a child has previously selected a mood. From the director's point of view, the statistics can only be displayed if a score is not currently being computed.

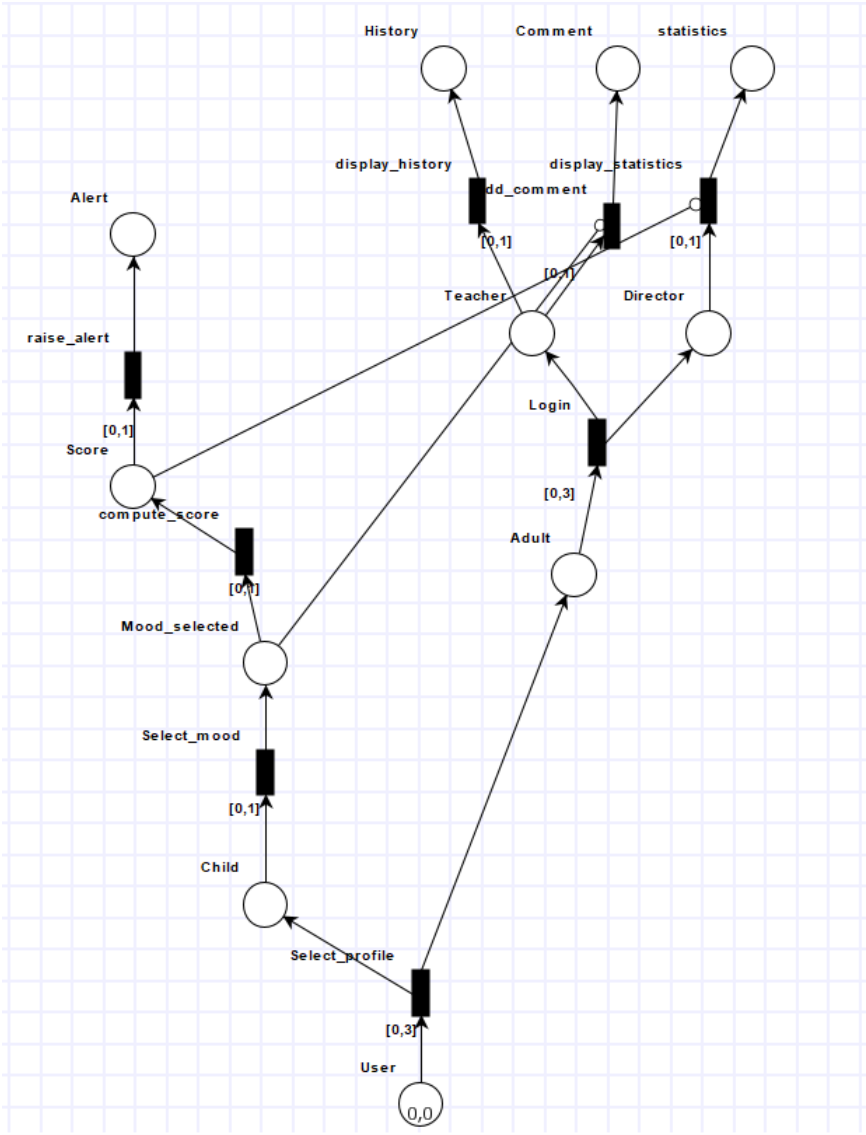

Figure 20 - TAPN diagram for How do I feel Today

3.3.8. Architecture diagram

The modelling type chosen was the architecture with layers as it seemed more relevant opposed to the client-server architectures or microservices, due to the size of the project. Four layers are enough for each to have a well-defined role, and respect hierarchy dependences.

The first handles all interaction with end users, including children, workers, and director.

The second one implements the core functionalities and rules of the application: mostly the calculations and computing.

The basic services layers abstract away technical operations and provide support to the logic layer. For instance, the CRUD operations (Create, Read, Update, Delete) that emphasizes the interactions with the databases.

Timely, the last layer is the databases layer. We divided in four main databases the data the system needs to store to be effective: user data, alerts, mood entries and comments.

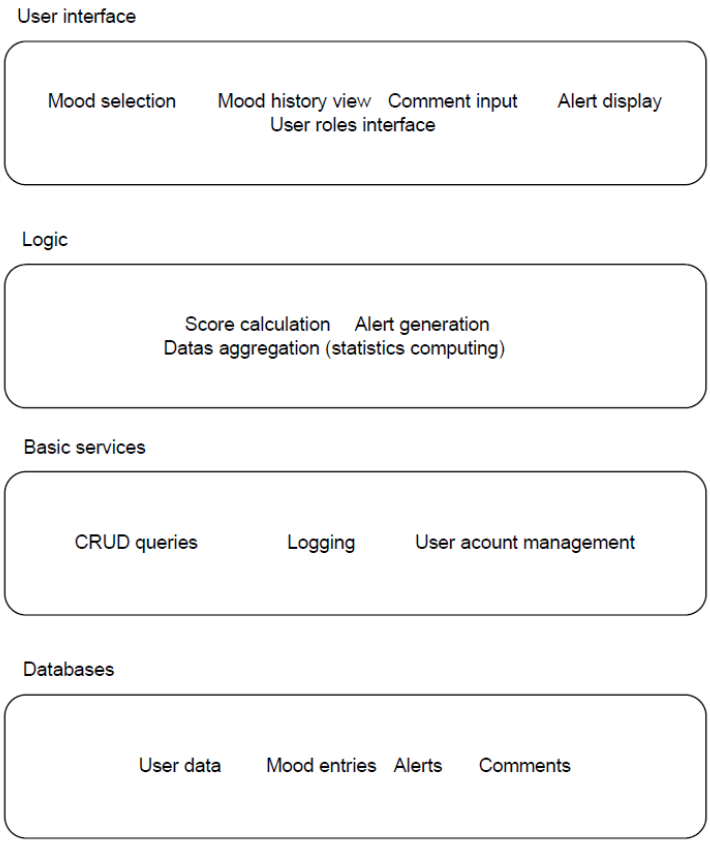

Figure 21 - Architecture for How do I feel Today

3.4. Testing approach/strategy (test-suite)

We used ChatGPT 3.5 and chat GPT4-o to generate the test suite. The prompt was the following:

We first gave full context of the project, diagrams included and asked to choose the most relevant way of testing between those seen in the lectures, having chosen first boundary values and equivalent classes. This posed and issue since the AI was making up requirements, having suggested it to change for equivalence testing only. When out of credits for the free version of GPT4-o, we copied the whole queries and outputs into a new chat, without the documents. Finally, we requested the generation of the equivalent classes and the corresponding tests. The output considered the length of the comments, which was not a requirement for the project and was therefore excluded.

ID	Equivalence Class	Type	Example(s)	Expected Result
EC1	Valid mood emoji	Valid	😊, 🌈, 😢, 😡	Mood recorded successfully
EC2	No mood selected	Invalid	null, ""	Error: "No mood selected"
EC3	Duplicate mood selection in one day	Invalid	😊 then 😢 on the same day	Error: "Only one mood per day allowed"
EC4	Child with mood history	Valid	Child A with 5 mood entries	History displayed
EC5	Child with no mood history	Invalid	Newly registered child	Message: "No data available"
EC6	Invalid child ID	Invalid	Nonexistent child ID	Error: "Child not found"
EC7	Comment on non-existent mood	Invalid	Trying to comment on a day with no mood	Error: "Cannot add comment"
EC8	Negative mood threshold not exceeded	Valid	2 moods 😢 or 😡 in past 7 days	No alert
EC9	Negative mood threshold exceeded	Valid	5+ moods 😢 or 😡 in past 7 days	Alert sent to educator
EC10	Authorized access to stats	Valid	Role = Director	Stats displayed
EC11	Unauthorized access to stats	Invalid	Role = Educator	Error: "Access denied"
EC12	Valid mood data available	Valid	Mood data for 200 children	Statistics generated
EC13	No mood data collected	Invalid	Empty mood database	Message: "Not enough data to generate"

Table 10 - Test suits for How do I feel Today

3.4.1. Discussion on the topic

1. ***Were the resulting tests helpful for the refinement of your product artifacts? Assume your project ideas will be developed. Is your AI-generated test suite suitable for testing?***

The test suite was accurate regarding our artifacts, and the AI use was truly efficient, even though the whole context and a little guidance were necessary. Some issues or details may have been forgotten, but in conclusion the test suite allow to test the well-functioning of the main features.

2. ***Will you use again AI tools to generate tests in other contexts?***

That being the goal, AI fulfilled his role, and we would most definitely use again AI to generate tests in another context.

3. ***If other students have to do the same task for the first time, what advice would you give? What are your lessons learned?***

The accuracy of the main prompt is very important, and you shouldn't hesitate to provide as much context as needed. Also, the AI can (at least for now) give false trails or answers. That's why one must use critical thinking and compare his expectations to the given output, and, if necessary, to ask for changes. Also, if the conversation is stuck because of a semantic gap or miscomprehension from the AI-tool, one can change tool. There exists a big enough diversity of those.

4. Lessons learned

This project's development was a worthwhile learning process every step of the way, technically and team-wise. The most significant lesson learned might be the value of doing a quality first analysis. While iterative improvement is a central principle of agile development, misunderstanding the front-end requirements far too frequently causes multiple iterations, model incompatibilities, and subsequent phase delays. Early and deep insight into the problem space finally leads to time and effort savings and delivers an integrated and accurate system design.

As great was the return on the value of the structured and coordinated collaboration. The rotation of the Scrum Master gave every member of the team a chance to manage the activities from different perspectives, and the splitting into sub-teams enabled us to focus on the separate modules and advance in parallel.

Another vital takeaway was the importance of adopting user cantered designs. By grounding our system architecture on detailed user stories, we ensured that the solution directly dealt with real needs and expectations of its intended users essential not only for system function alignment with true workflows in the real world but also for usability as well as user involvement, a critical imperative to all projects.

We also gained critical through critical scrutiny of AI tools in their application for test case generation and documentation enhancement. Their output needed to be critically validated, confirming the importance of critical human oversight for ensuring accuracy and relevance in all outputs.

Lastly, the project underscored the fact that model-driven development demands high consistency and iteration. Although UML, EFSM, TAPN, and OCL were instrumental in enhancing the lucidity and organization of our design, it was

difficult to maintain consistency across all the diagrams and logic models. Frequent peer inspection and validation sessions were necessary to preserve internal coherence during the project cycle.

5. Conclusion and future work

This project addressed a complex and socially relevant challenge through the improvement of logistic processes and emotional monitoring in a childcare environment through a model-based software solution, seeking to reduce administrative burden while supporting the well-being of both children and staff, through a grounding real-life scenario making it more engaging and motivating for the entire team.

The outcome is an exhaustive and coherent set of software engineering outputs: user stories, UML models, formal constraints (OCL), behavioral models (EFSM, TAPN), early interface mockups, and a thoroughly structured test suite. These artifacts taken together form a good foundation for subsequent development and possible implementation.

In summary, the project was a comprehensive and very applied exercise in model-based software engineering techniques. It offered an opportunity to apply theoretical learning to a genuine and valuable problem area, strongly reinforcing key skills in system analysis, user friendly design, collaborative development, and critical evaluation. It was therefore a significant part of the academic and professional growth of the project team.

6. Annexes

User Story	Test Case ID	Test Case Description	Type of Test	Steps	Expected Results
User Story 1: As a childcare worker I want to be able to request/inform when I will be absent so that everybody knows I won't be at my Workplace.	TC 1.1	Submit New Absence Request (Valid Data)	Positive	1. Childcare worker logs in. 2. Navigate to the "Submit Absence Request" feature. 3. Enter valid start and end dates for absence. 4. Select a valid reason for absence.	System confirms successful submission. Request status is "pending". Director can view the request.

				5. Click "Submit" or "Request Absence".	
	TC 1.2	Submit New Absence Request (Missing Required Fields)	Negative	1. Childcare worker logs in. 2. Navigate to the "Submit Absence Request" feature. 3. Attempt to submit the form with one or more required fields left blank (no start date).	System displays an error message indicating missing required fields and does not submit the request.
User Story 8: As a childcare worker I want to update or cancel an absence request, so I can respond to changes in my personal situation.	TC 8.1	Update Pending Absence Request (Valid Change)	Positive	1. Childcare worker logs in. 2. Navigate to "View Requests" to see existing requests. 3. Select a pending absence request for modification. 4. Change a detail (end date, reason). 5. Save the changes.	The system updates the absence request. The Director receives an alert about the change.
	TC 8.2	Cancel Pending Absence Request	Positive	1. Childcare worker logs in. 2. Navigate to "View Requests". 3. Select a pending absence request for cancellation. 4. Initiate the cancellation process. 5. Confirm cancellation if prompted.	The system cancels the absence request. The request no longer appears as pending for the childcare worker, and the Director is notified, or the request is removed from their view.
User Story 11: As the director I want to view all submitted vacation requests from staff (with detailed info), so I	TC 11.1	View All Pending Requests	Positive	1. Director logs in. 2. Navigate to the "View Requests" section.	A list of all pending absence requests from staff is displayed,

can review and manage them efficiently.					including detailed information (staff name, dates, reason, status).
	TC 11.2	View Approved Requests	Positive	1. Director logs in. 2. Navigate to the "View Requests" section. 3. Apply a filter (if available) to show approved requests.	A list of all approved absence requests from staff is displayed, including detailed information.
User Story 12: As the director I want to approve or reject vacation requests, so I can ensure there is enough staff available.	TC 12.1	Approve a Pending Request	Positive	1. Director logs in. 2. Navigate to "View Requests". 3. Select a pending request. 4. Choose the "Approve" option.	The request status changes to "Approved". The childcare worker who submitted the request is notified of the approval.
	TC 12.2	Reject a Pending Request	Positive	1. Director logs in. 2. Navigate to "View Requests". 3. Select a pending request. 4. Choose the "Reject" option.	The request status changes to "Rejected". The childcare worker who submitted the request is notified of the rejection.
User Story 16: As the director, I want to filter vacation requests by role or group (preschool, nursery), so I can balance leave within each department.	TC 16.1	Filter Requests by Role/Group (Preschool)	Positive	1. Director logs in. 2. Navigate to "View Requests". 3. Apply a filter for "Role: Preschool" or "Group: Preschool".	Only absence requests submitted by staff members belonging to the "Preschool" role/group are displayed.
User Story 17: As the director, I want to export vacation plans or summaries for documentation or	TC 17.1	Export All Vacation Plans	Positive	1. Director logs in. 2. Navigate to the export feature (	A file containing the vacation plans or summaries is

internal reporting purposes.				"Export Plans" or "Generate Report"). 3. Select to export all vacation plans. 4. Choose an export format (if options are available, PDF, CSV). 5. Initiate the export.	successfully generated and downloaded or presented for download. The content accurately reflects the current vacation data.
	TC 17.2	Export Filtered Vacation Plans	Positive	1. Director logs in. 2. Apply a filter to the vacation requests (show only "Preschool" requests). 3. Navigate to the export feature. 4. Initiate the export of the filtered data.	A file containing only the filtered vacation plans is successfully generated and downloaded. The content accurately reflects the filtered data.
User Story 18: As an authorised personal I want to be able to modify the planning if an update to a request occurs so that the planning is up to date.	TC 18.1	Modify Planning After Request Update (Approved Request)	Positive	1. An approved request is updated by a childcare worker. 2. Authorized Personnel logs in. 3. Navigate to the planning view (calendar). 4. Locate the entry corresponding to the updated request. 5. Modify the planning to reflect the change.	The Authorized Personnel is able to modify the planning. The updated planning is correctly displayed and saved. The system may highlight the change or prompt for an update to the planning.
	TC 18.2	Modify Planning After Request Approval	Positive	1. A new absence request has just been approved by the Director. 2. Authorized Personnel logs in.	The Authorized Personnel is able to add the approved absence to the planning. The planning reflects the

				3. Navigate to the planning view. 4. Add the approved absence to the planning.	newly approved absence.

Table 1111- Text suites for Vacation Planning